JSPM's
Rajarshi Shahu College of Engineering, Tathawade, Pune-33
(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)
Department of Information Technology
Academic Year 2026-27

DBMS Mini Project

Project Title : Food Delivery System

Team details

Sr. No.	Roll No.	Name of student
1	IT3253	Varad Wakchaure
2	IT3254	Harshal Yadav
3	IT3257	Raghav Wadekar
4	IT3275	Siddhant Bhandare

JSPM's
Rajarshi Shahu College of Engineering, Tathawade, Pune-33
(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)
Department of Information Technology
Academic Year 2026-27

INDEX

Sr. No.	Section Title	Page No.
1	Introduction	3
2	Project Overview	3
3	System Architecture	4
4	Technology Stack	5
5	Database Design	6
6	MySQL Database (AWS RDS)	7
7	MongoDB Database (Atlas)	9
8	Key SQL Queries & Database Objects	10
9	Application Features & User Roles	12
10	System Screenshots	13
11	Database Configuration Screenshots	14
12	Conclusion	15

1. Introduction

In today's rapidly evolving digital landscape, food delivery platforms have become an essential part of modern life. Building such a platform requires careful thought about data management, as different types of data have different storage and retrieval needs. This project is a mini project for the Database Management Systems (DBMS) course, designed to demonstrate the practical application of both relational and non-relational database systems in a single, real-world application.

The Food Delivery System is a full-stack web application that uses a Hybrid Database Architecture — combining MySQL (a relational database) and MongoDB (a NoSQL document database). This approach mirrors what major food delivery platforms such as Zomato and Swiggy use in production, where different types of data are stored in the most appropriate database system.

The system demonstrates key DBMS concepts including normalization, transactions, triggers, stored procedures, views, indexes, aggregation pipelines, and schema validation — all in the context of a working, functional application.

2. Project Overview

The Food Delivery System is a complete web application with four distinct user roles: Customer, Restaurant Owner, Delivery Partner, and Admin. Each role has its own dedicated dashboard with role-specific features. The system handles the entire lifecycle of a food order — from browsing restaurants to placing an order, restaurant acceptance, delivery assignment, and final delivery confirmation.

Project Objectives

- Demonstrate the use of a Hybrid Database Architecture using MySQL and MongoDB.
- Implement ACID-compliant transactions for order and payment processing.
- Use MongoDB's flexible document model for restaurant menus and event logs.
- Apply advanced SQL concepts: Triggers, Views, Stored Procedures, and Indexes.
- Build a fully functional, role-based full-stack web application.
- Configure and use cloud-hosted databases: AWS RDS (MySQL) and MongoDB Atlas.

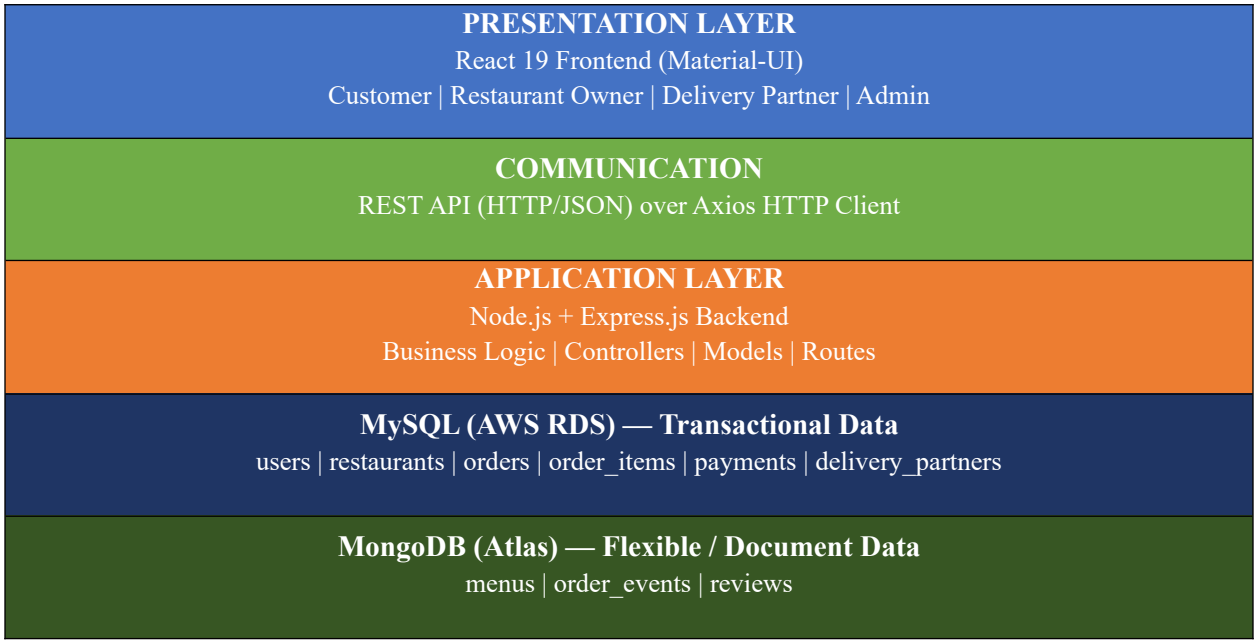
Project Statistics

Metric	Details
Frontend Pages	5 Pages (Landing + 4 Role Dashboards)
Backend API Endpoints	43+ REST API Endpoints
MySQL Tables	6 Tables (Normalized to 3NF)
MongoDB Collections	3 Collections (menus, order_events, reviews)
SQL Triggers	1 Trigger (cancel_order_on_refund)
SQL Views	3 Views (restaurant_revenue, top_customers, active_deliveries)
Stored Procedures	2 (assign_partner, complete_delivery)
Total Lines of Code	5000+ lines

3. System Architecture

The system follows a standard three-tier architecture: a React-based Frontend, a Node.js + Express Backend, and a dual-database layer consisting of MySQL (hosted on AWS RDS) and MongoDB (hosted on MongoDB Atlas). The architecture is designed so that each type of data is stored in the most suitable database.

Architecture Overview



Data Flow: Placing an Order

When a customer places an order, the system coordinates between both databases as follows:

- Step 1: Customer selects items from the restaurant menu (data fetched from MongoDB).
- Step 2: The backend validates that the selected menu items exist in MongoDB.
- Step 3: A MySQL Transaction begins (BEGIN TRANSACTION).
- Step 4: The order record is inserted into the 'orders' table.
- Step 5: Individual items are inserted into the 'order_items' table.
- Step 6: A payment record is inserted into the 'payments' table.
- Step 7: The transaction is committed (COMMIT). If any step fails, ROLLBACK is triggered.
- Step 8: An event log document (status: PLACED) is written to the 'order_events' collection in MongoDB.

- Step 9: A success response is returned to the customer's browser.

Why Hybrid Architecture?

Data Type	Database Used	Reason
Users, Orders, Payments	MySQL (AWS RDS)	Requires ACID transactions and strong consistency
Restaurant Menus	MongoDB (Atlas)	Nested categories and customizations fit a document model
Order Event Logs	MongoDB (Atlas)	Append-only time-series data; fast writes, no joins needed
Reviews & Ratings	MongoDB (Atlas)	Flexible schema; different restaurants may need different fields
Delivery Partners	MySQL (AWS RDS)	Relational data linked to orders; requires referential integrity

4. Technology Stack

The project is built using a modern, industry-standard technology stack. The choice of technologies ensures scalability, developer productivity, and alignment with what is used in real-world food delivery applications.

Backend Technologies

Technology	Version	Purpose
Node.js	18.x	JavaScript runtime for the server-side application
Express.js	4.x	Web framework for building REST API endpoints
mysql2	Latest	MySQL driver for Node.js with promise/async support
mongodb	Official Driver	MongoDB driver for Node.js to query Atlas collections
dotenv	Latest	Environment variable management for secure config
cors	Latest	Enables Cross-Origin Resource Sharing for the frontend

Frontend Technologies

Technology	Version	Purpose
React	19.x	UI library for building the single-page application
Material-UI (MUI)	5.x	React component library for a professional UI design
React Router	v6	Client-side routing between the four dashboards
Axios	Latest	HTTP client for making API calls to the backend
Recharts	Latest	Charting library used in the Admin analytics dashboard

Database & Cloud Technologies

Technology	Service	Purpose
MySQL 8.0	AWS RDS (Free Tier)	Relational database for transactional data
MongoDB 8.0	MongoDB Atlas (M0 Free Cluster)	NoSQL document database for flexible schema data
DBeaver 26.x	Desktop Application	SQL client for managing and querying the MySQL database
MongoDB Compass	Desktop Application	GUI tool for browsing and querying MongoDB collections
AWS Console	Cloud Management	Managing the RDS MySQL instance and network security
MongoDB Atlas UI	Cloud Management	Managing the Atlas cluster, users, and network access

5. Database Design

The database design follows the principles of relational database normalization and NoSQL document modeling. The MySQL schema is normalized to the Third Normal Form (3NF) to eliminate data redundancy and ensure data integrity. The MongoDB collections are designed to take advantage of the document model for storing hierarchical and flexible data.

6. MySQL Database (AWS RDS)

MySQL is used for all transactional data that requires ACID (Atomicity, Consistency, Isolation, Durability) guarantees. The database is hosted on Amazon Web Services RDS using a db.t3.micro instance (Free Tier) in the ap-south-1 (Mumbai) region, running MySQL Community Edition 8.0.

Database Schema: Tables & Relationships

The MySQL database 'food_delivery' consists of 6 normalized tables:

Table: users

Stores all system users (customers, restaurant owners, delivery partners, admins). Central table linked to all other entities.

Columns: *user_id (PK), name, email, phone, role, created_at*

Table: restaurants

Stores restaurant information. Each restaurant is owned by a user with role='owner'.

Columns: *restaurant_id (PK), owner_id (FK→users), name, city, address, cuisine_type, is_open*

Table: delivery_partners

Stores delivery partner details. Linked one-to-one with the users table.

Columns: *partner_id (PK), user_id (FK→users, UNIQUE), vehicle_type, is_available*

Table: orders

Core transactional table. Records every order placed. Links customers, restaurants, and delivery partners.

Columns: *order_id (PK), user_id (FK), restaurant_id (FK), partner_id (FK), total_amount, status, delivery_address, placed_at*

Table: order_items

Stores individual items within each order. Item name and price are intentionally denormalized to preserve historical order records.

Columns: *item_id (PK), order_id (FK), menu_item_id (MongoDB ObjectId), item_name, unit_price, quantity, customizations*

Table: payments

Records payment information for each order. Has a one-to-one relationship with orders.

Columns: *payment_id (PK), order_id (FK, UNIQUE), amount, method, status, paid_at*

Order Status Flow

The 'status' field in the orders table uses a MySQL ENUM type and follows this lifecycle:

`PLACED` → `ACCEPTED` → `PREPARING` → `READY_FOR_PICKUP` → `PICKED_UP` → `DELIVERED`

Orders can also be set to REJECTED or CANCELLED (triggered automatically when a payment is refunded).

Normalization (3NF)

The schema is in Third Normal Form (3NF). No table has transitive dependencies. For example, restaurant details (name, city, cuisine) are stored in the 'restaurants' table, not duplicated in the 'orders' table. User details are in 'users', not copied to 'restaurants' or 'orders'. The only intentional denormalization is item_name and unit_price in 'order_items', which preserves historical order data even if menu prices change later.

7. MongoDB Database (Atlas)

MongoDB is used for data that has a flexible, hierarchical, or schema-variable structure. The database is hosted on MongoDB Atlas, using a free M0 cluster named 'FoodDeliveryCluster' in the AWS Mumbai (ap-south-1) region, running MongoDB version 8.0.34.

Collections Overview

Collection	Purpose	Key Fields
menus	Stores the complete menu for each restaurant with nested categories and items.	restaurant_id, categories[] → items[] → name, price, veg, customizations[]
order_events	Stores the event log (audit trail) for every order status change.	order_id, status, timestamp, by_role, note
reviews	Stores customer reviews and ratings for restaurants and delivery partners.	order_id, user_id, restaurant_id, restaurant_rating, delivery_rating, comment

Menu Document Structure

The menus collection uses MongoDB's embedded document model to store nested categories and items. This is ideal because a restaurant's menu is always accessed as a complete unit, and embedding avoids expensive JOIN operations. Each menu document contains:

Sample Menu Document (MongoDB Compass View):

```
{
  "restaurant_id": 1,
  "categories": [
    {
      "name": "Pizzas",
      "items": [
        {
          "_id": ObjectId("..."),
          "name": "Margherita Pizza",
          "price": 249,
          "veg": true,
          "description": "Classic pizza with tomato sauce and mozzarella",
          "image": "https://images.unsplash.com/..",
          "customizations": [
            { "name": "Extra Cheese", "price": 50 },
            { "name": "Thin Crust", "price": 0 }
          ]
        }
      ]
    }
  ]
}
```

MongoDB Indexes

Indexes are created on all frequently queried fields for optimal query performance:

Collection	Index	Type	Purpose
menus	restaurant_id	Single Field (ASC)	Fast lookup of menu by restaurant
menus	categories.items.name	Text Index	Full-text search across menu items
order_events	order_id	Single Field (ASC)	Quickly retrieve all events for an order
order_events	timestamp	Single Field (DESC)	Sorted event timeline retrieval
reviews	restaurant_id + created_at	Compound Index	Paginated reviews sorted by date

8. Key SQL Queries & Database Objects

This section presents the most important SQL constructs implemented in the project, demonstrating key DBMS concepts: Triggers, Views, Stored Procedures, Transactions, and Joins.

8.1 Trigger: cancel_order_on_refund

This AFTER UPDATE trigger on the 'payments' table automatically cancels an order when its payment status is changed to 'REFUNDED'. This demonstrates automated database logic that runs without any application code change.

SQL Trigger:

```
CREATE TRIGGER cancel_order_on_refund
AFTER UPDATE ON payments
FOR EACH ROW
BEGIN
  IF NEW.status = 'REFUNDED' AND OLD.status != 'REFUNDED' THEN
    UPDATE orders
    SET status = 'CANCELLED'
    WHERE order_id = NEW.order_id;
  END IF;
END;
```

8.2 View: restaurant_revenue

This view simplifies the complex JOIN and aggregation query needed to calculate each restaurant's total revenue. Instead of writing the full query each time, the Admin dashboard simply queries SELECT * FROM restaurant_revenue.

SQL View — restaurant_revenue:

```
CREATE VIEW restaurant_revenue AS
SELECT
  r.restaurant_id,
  r.name          AS restaurant_name,
  r.city,
  r.cuisine_type,
  COUNT(o.order_id)          AS total_orders,
  COALESCE(SUM(o.total_amount), 0)  AS total_revenue,
  COALESCE(AVG(o.total_amount), 0)  AS avg_order_value
FROM restaurants r
LEFT JOIN orders o
  ON r.restaurant_id = o.restaurant_id
```

```
  AND o.status = 'DELIVERED'
GROUP BY r.restaurant_id, r.name, r.city, r.cuisine_type
ORDER BY total_revenue DESC;
```

8.3 MySQL Transaction: Place Order

Every order placement is wrapped in a MySQL transaction to ensure ACID properties. If any step fails (e.g., inserting order items or payment), the entire transaction is rolled back, preventing partial data.

ACID Transaction for Order Placement:

```
-- BEGIN TRANSACTION (implemented in Node.js orderModel.js)
START TRANSACTION;


  -- Step 1: Insert the order
  INSERT INTO orders (user_id, restaurant_id, total_amount, status,
delivery_address)
  VALUES (3, 2, 748.00, 'PLACED', '123 Main St, Mumbai');


  -- Step 2: Insert order items
  INSERT INTO order_items (order_id, menu_item_id, item_name, unit_price, quantity)
  VALUES (LAST_INSERT_ID(), '6ad7e84...17c', 'Margherita Pizza', 249, 2);


  INSERT INTO order_items (order_id, menu_item_id, item_name, unit_price, quantity)
  VALUES (LAST_INSERT_ID(), '6ad7e84...17d', 'Garlic Bread', 99, 1);


  -- Step 3: Record payment
  INSERT INTO payments (order_id, amount, method, status)
  VALUES (LAST_INSERT_ID(), 748.00, 'UPI', 'SUCCESS');


COMMIT;  -- If any step above fails, ROLLBACK is triggered instead
```

8.4 JOIN Queries for Order Details

INNER JOIN Query — Order Details:

```
-- Get full order details with restaurant and customer info
SELECT
  o.order_id, o.status, o.total_amount, o.placed_at,
  u.name AS customer_name, u.phone AS customer_phone,
  r.name AS restaurant_name, r.city
FROM orders o
INNER JOIN users u ON o.user_id = u.user_id
INNER JOIN restaurants r ON o.restaurant_id = r.restaurant_id
WHERE o.status = 'PLACED'
```

```
ORDER BY o.placed_at DESC;
```

8.5 MongoDB Aggregation Pipeline

The Admin dashboard uses a MongoDB aggregation pipeline to compute peak order hours from the order_events collection:

MongoDB Aggregation — Peak Order Hours:

```
db.order_events.aggregate([
  { $match: { status: "PLACED" } },
  { $project: { hour: { $hour: "$timestamp" } } },
  { $group: { _id: "$hour", order_count: { $sum: 1 } } },
  { $sort: { order_count: -1 } }
])
```

9. Application Features & User Roles

The application supports four distinct user roles, each with a dedicated dashboard. Users select their role on the landing page and are directed to the appropriate interface.

Customer Dashboard

- Browse all available restaurants with search functionality.
- View full restaurant menus (fetched from MongoDB Atlas).
- Add items to a shopping cart with customizations.
- Place orders using mock payment methods (UPI, Card, COD, Wallet).
- Track order status in real-time with a timeline view.
- View complete order history.

Restaurant Owner Dashboard

- View all incoming orders for their restaurant.
- Accept or reject incoming orders (status: PLACED).
- Update order status to PREPARING when food preparation begins.
- Mark order as READY_FOR_PICKUP when food is ready.
- View restaurant statistics: total orders, total revenue, average order value.
- Filter orders by status for easy management.

Delivery Partner Dashboard

- View all available deliveries (orders in READY_FOR_PICKUP status).

- Accept a delivery — triggers the assign_partner stored procedure.
- Update order status to PICKED_UP and then DELIVERED.
- View current active deliveries and earnings.
- Track delivery fee (10% of order value).

Admin Dashboard

- View system-wide statistics: total restaurants, orders, users, revenue.
- See top-rated restaurants (data from MongoDB aggregation).
- View restaurant revenue table (data from MySQL restaurant_revenue view).
- Monitor recent orders and system activity.
- Analyze peak order hours using MongoDB aggregation charts.

10. System Screenshots

The following screenshots show the working application with all four user dashboards and the landing page of the Food Delivery System.

Customer Dashboard

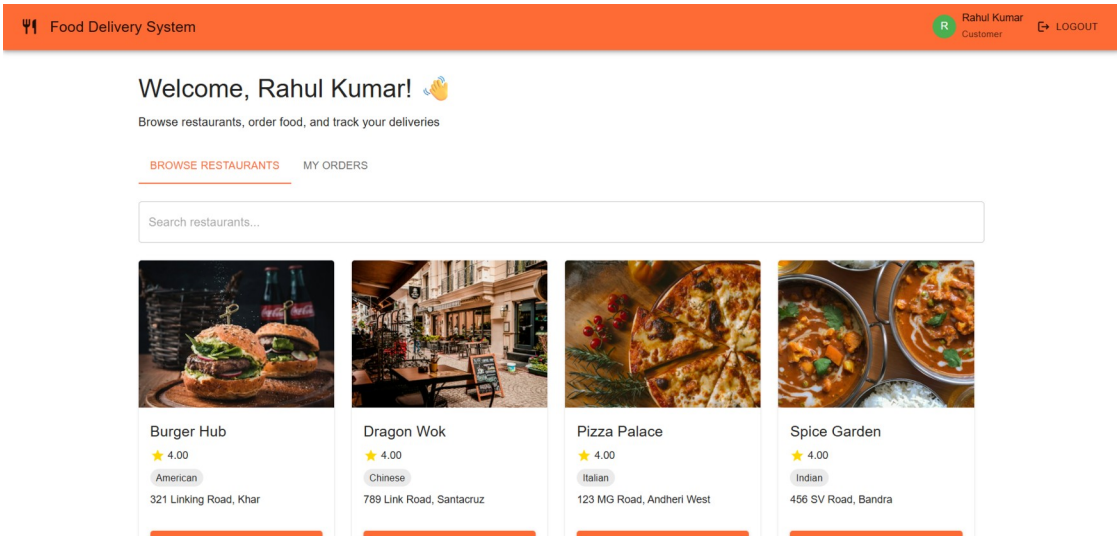


Figure 1: Customer Dashboard — Browse Restaurants (showing Burger Hub, Dragon Wok, Pizza Palace, Spice Garden)

Restaurant Owner Dashboard

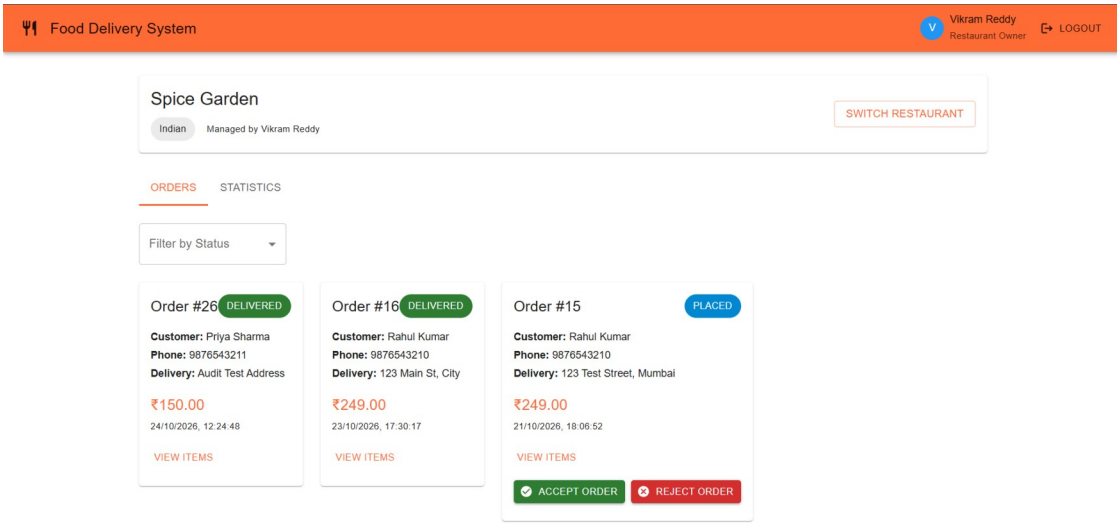


Figure 2: Restaurant Owner Dashboard — Order Management (Spice Garden, managed by Vikram Reddy)

Delivery Partner Dashboard

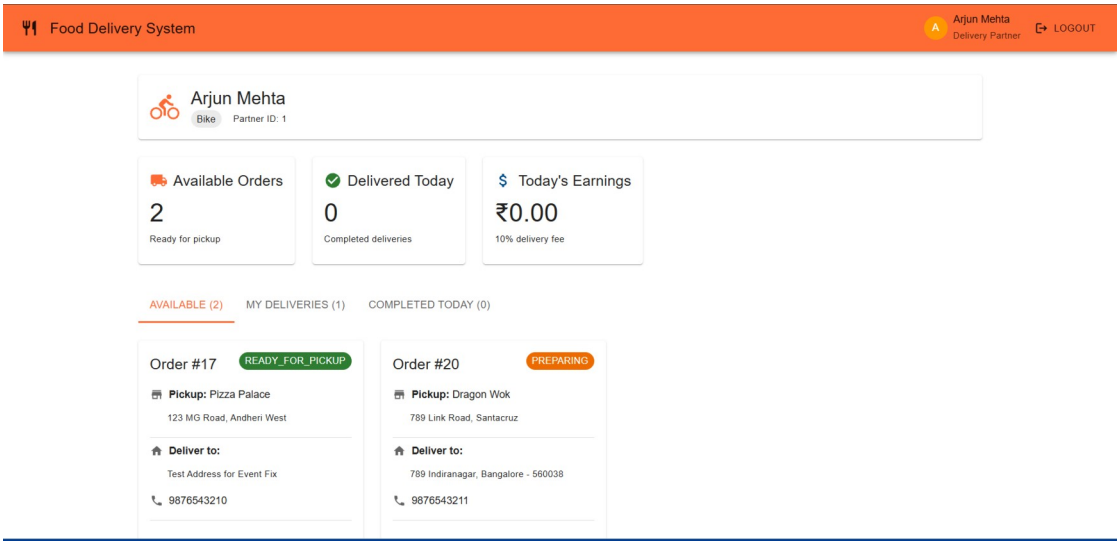


Figure 3: Delivery Partner Dashboard — Available Deliveries (Arjun Mehta, Bike Partner)

Admin Dashboard

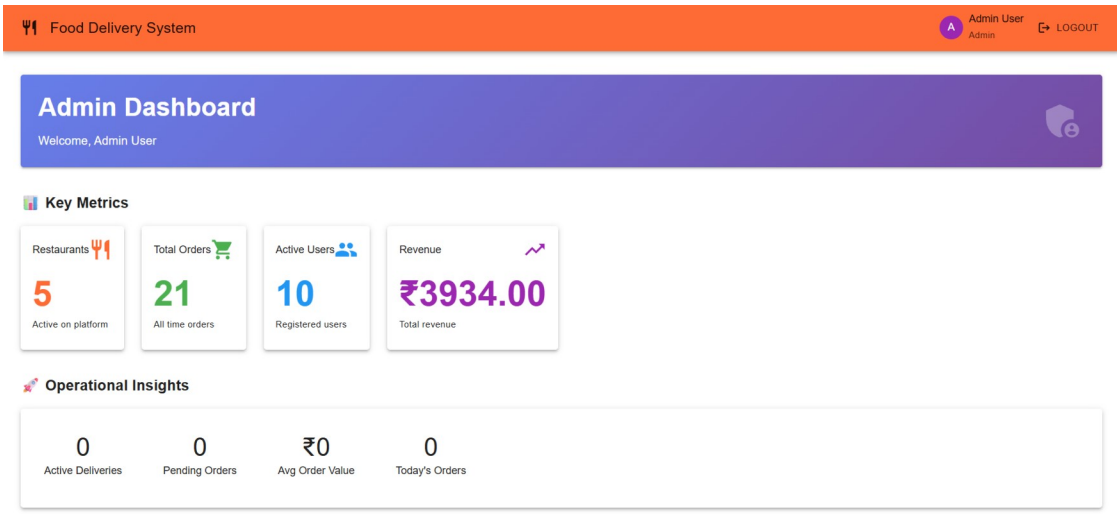


Figure 4: Admin Dashboard — Key Metrics (5 Restaurants, 21 Total Orders, 10 Users, ₹3934 Revenue)

11. Database Configuration Screenshots

This section shows the cloud-based database configurations and the GUI tools used to manage and interact with the databases.

AWS RDS — MySQL Configuration

The MySQL database is hosted on Amazon Web Services RDS. The instance identifier is 'food-delivery-db', running MySQL Community Edition on a db.t3.micro instance in the Asia Pacific (Mumbai) ap-south-1a region. The status shows 'Available' with 2 active connections.

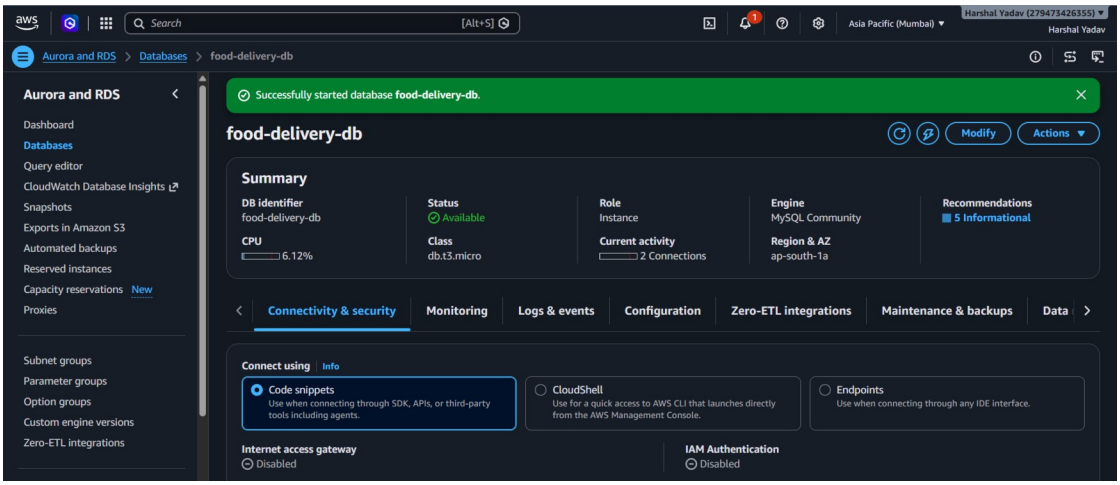


Figure 5: AWS RDS Console — food-delivery-db (MySQL 8.0, db.t3.micro, ap-south-1a)

DBeaver — MySQL Database View

DBeaver is used as the SQL client to connect to the AWS RDS instance, run queries, view table data, manage triggers, views, procedures, and verify the schema. The screenshot shows the 'order_items' table with live data from the food_delivery database.

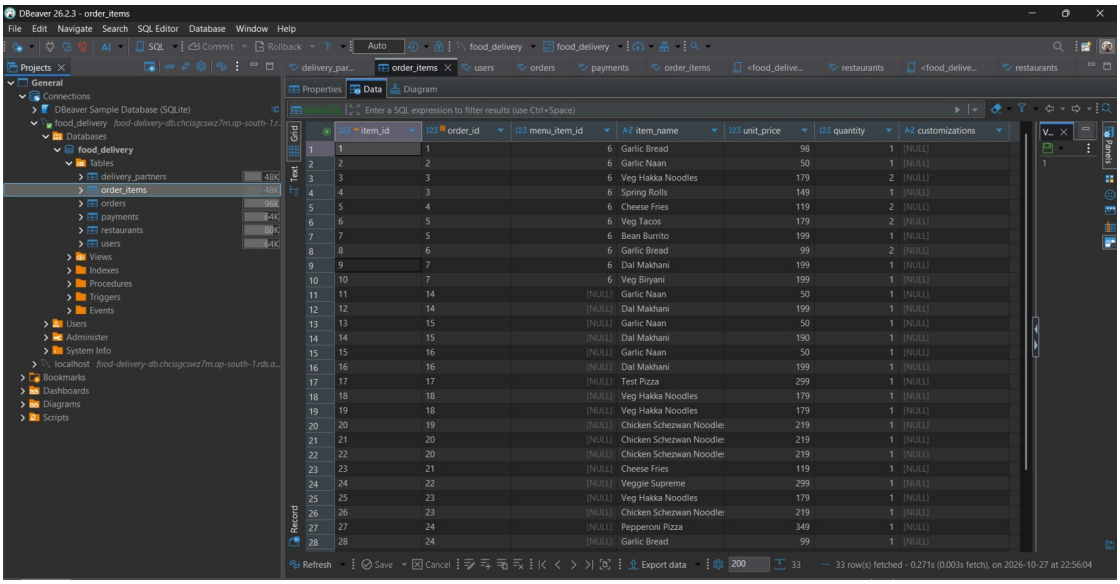


Figure 6: DBeaver 26.2.3 — food_delivery MySQL Database (AWS RDS endpoint), showing order_items table

MongoDB Atlas — Cluster Overview

The MongoDB database is hosted on MongoDB Atlas. The cluster name is 'FoodDeliveryCluster', running MongoDB version 8.0.34 on the AWS Mumbai (ap-south-1) region using the M0 Free Tier. The cluster uses a sharded configuration with one primary and two secondary nodes.

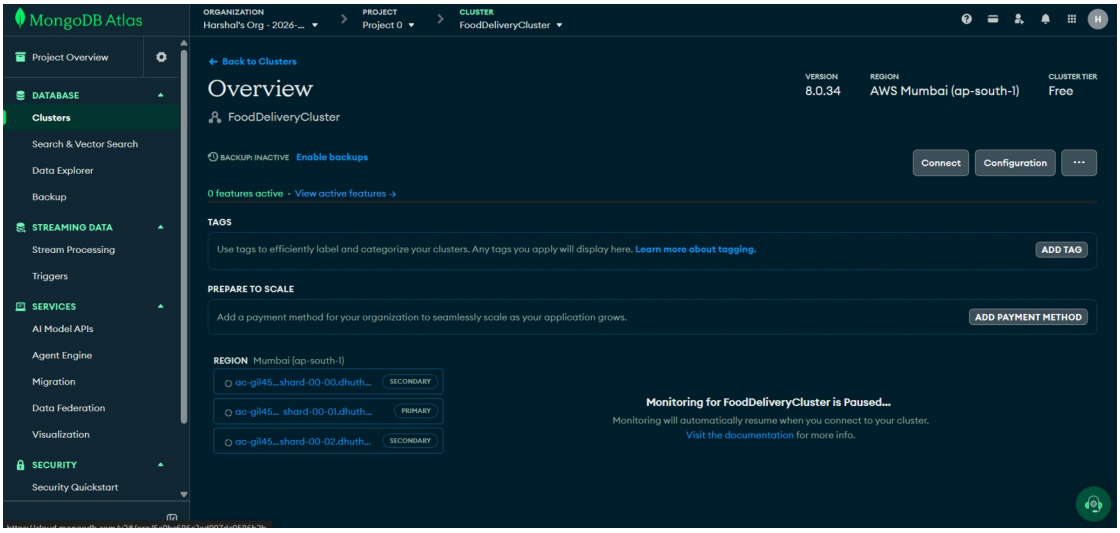


Figure 7: MongoDB Atlas — FoodDeliveryCluster Overview (v8.0.34, AWS Mumbai, Free Tier)

MongoDB Compass — Collection View

MongoDB Compass is used to browse and query MongoDB collections. The screenshot shows the 'menus' collection in the 'food_delivery' database, displaying the embedded document structure with nested categories and menu items including name, price, veg flag, description, image, and customizations.

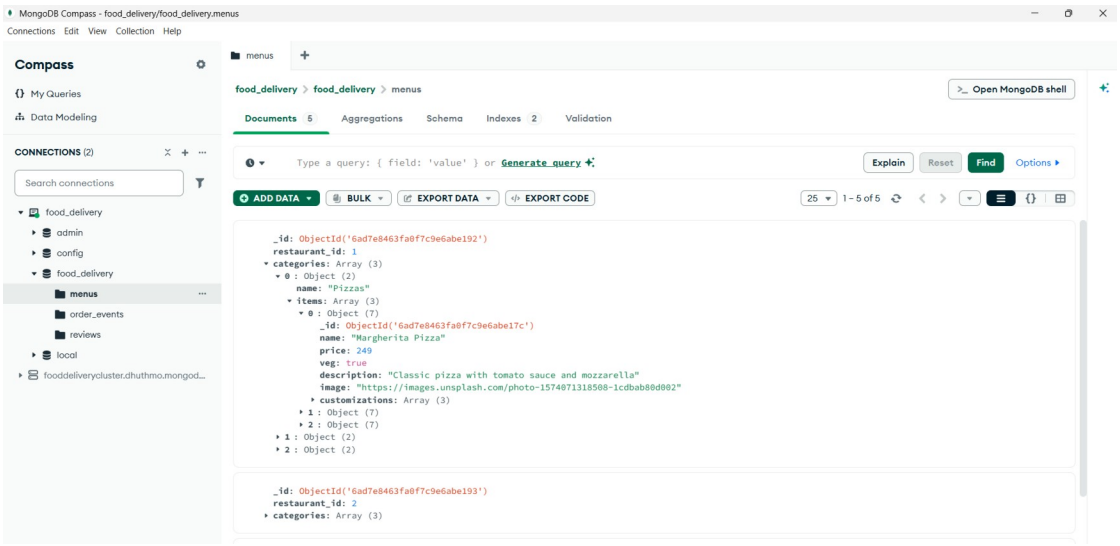


Figure 8: MongoDB Compass — food_delivery.menus collection (showing embedded menu documents)

12. Conclusion

The Food Delivery System successfully demonstrates the design, development, and deployment of a full-stack web application using a Hybrid Database Architecture. By combining MySQL for transactional data and MongoDB for flexible schema data, the project shows how to use the right database for the right type of data — a pattern widely used in modern software engineering.

The project covers a comprehensive set of DBMS concepts, including relational schema design and normalization (3NF), ACID transactions, database triggers, views, stored procedures, foreign key constraints, indexes, MongoDB document modeling, aggregation pipelines, and schema validation. All these concepts are demonstrated in the context of a real, working application rather than in isolation.

The system is hosted on cloud services — AWS RDS for MySQL and MongoDB Atlas — making it production-ready and demonstrating real-world deployment practices. GUI tools such

as DBeaver and MongoDB Compass were used to manage and verify the database state during development.

Key Learnings

- Selecting the correct database type (relational vs. document) is crucial for application performance and maintainability.
- ACID transactions in MySQL ensure that order placements are always complete or completely rolled back — never partial.
- MongoDB's embedded document model is highly efficient for hierarchical data like restaurant menus.
- Stored procedures and triggers encapsulate business logic at the database level, making the application more robust.
- Cloud database services (AWS RDS, MongoDB Atlas) simplify database administration with built-in backups, monitoring, and security.
- A well-normalized schema prevents data anomalies and makes the system easier to maintain and extend.

Future Enhancements

- Add JWT-based authentication and session management for secure user login.
- Implement WebSocket connections for real-time order status push notifications.
- Integrate a real payment gateway (Razorpay or Stripe) for actual payment processing.
- Add Redis caching layer to reduce database load for frequently accessed data (e.g., restaurant listings).
- Deploy the full application to a cloud platform (AWS EC2 or Heroku) for public access.

Overall, this project serves as a practical demonstration of advanced DBMS concepts applied to a real-world problem domain, showcasing both theoretical knowledge and hands-on implementation skills.